Local DNS Server Report -- 12311043

My impletation and explanation

Task1.1 -- Automatically Detect Outbound Interface IP

- Requirement: Discover the best local IP used for outbound connections.
- Implementation: `get_local_ip()`
  - Creates a UDP socket, “connects” to 8.8.8.8:53 to let OS choose the egress interface, then reads `getsockname()[0]`.
  - On failure (e.g., no network), returns robust fallback "0.0.0.0".

```
def get_local_ip():
    # Heuristic: a UDP "connect" to 8.8.8.8:53 sends no packets but binds a local address; we read that chosen outbound IP.
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
            s.connect(('8.8.8.8', 53))
            local_ip = s.getsockname()[0]
            return local_ip
    except Exception:
        return '0.0.0.0'
```

Task1.2 -- DNSServer Implementation

- Requirement: Start/stop lifecycle, receiver-sender-worker threads, queue-based decoupling.
- Implementation:
  - `DNSServer.__init__`: create UDP socket, queues, worker pool metadata, CacheManager instance.
  - `start()`:
    - Binds socket, starts Receiver thread (`_receive_loop`) and Sender thread (`_send_loop`).
    - Spawns N worker threads (`DNSHandler`) and marks them daemon.
    - Main thread waits until interrupted, then gracefully shuts down.
  - `stop()`:
    - Sets `self.running=False`, broadcasts sentinel `(None, None)` to workers to exit, saves cache, closes socket.
  - `_receive_loop()`:
    - Blocking `recvfrom(512)` while running, pushes `(data, addr)` to request queue.

- `_send_loop()`:
  - Pops `(addr, response_bytes)` from response queue and `sendto()`.

```
class DNSServer:
    def __init__(self, source_ip, source_port, ip='127.0.0.1', port=5533,
num_workers=20):
    # TODO: Initialize core components for multi-threading architecture.
        self.source_ip = source_ip
        self.source_port = source_port
        self.ip = ip
        self.port = port
        # UDP socket with SO_REUSEADDR for smoother restarts.
        self.socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        # Control flags and structures for the thread model.
        self.running = False
        self.request_queue = Queue()
        self.response_queue = Queue()
        self.num_workers = num_workers
        self.workers = []
        self.cache_manager = CacheManager()

    def start(self):
    # TODO
        self.running = True
        try:
            self.socket.bind((self.ip, self.port))
            print(f"Starting DNS server on {self.ip}:{self.port}")
            # Receiver thread: only reads datagrams and enqueues raw messages.
            self.receiver = threading.Thread(target=self._receive_loop,
daemon=True, name="Receiver")
            self.receiver.start()
            # Sender thread: only dequeues and sends packed responses.
            self.sender = threading.Thread(target=self._send_loop, daemon=True,
name="Sender")
            self.sender.start()
            # Worker pool: each worker parses, applies policy, uses cache, and
resolves on cache miss.
            for i in range(self.num_workers):
                worker = DNSHandler(
                    self.source_ip,
                    self.source_port,
                    self.cache_manager,
                    self.request_queue,
                    self.response_queue,
```

```
                    worker_id=i
                )
                worker.daemon = True
                worker.start()
                self.workers.append(worker)
                print(f"Started worker thread {i}")
            print(f"DNS server started with {self.num_workers} workers")
            try:
                # Keep the main thread responsive to Ctrl+C while background
threads do the work.
                while self.running:
                    time.sleep(1)
            except KeyboardInterrupt:
                print("\nReceived interrupt signal, shutting down...")
        except Exception as e:
            # Startup failure should be surfaced but also trigger cleanup.
            print(f"Failed to start DNS server: {e}")
        finally:
            self.stop()

    def stop(self):
    # TODO
        print("Shutting down DNS server...")
        self.running = False
        # Ask workers to exit by pushing sentinel tuples.
        try:
            for _ in self.workers:
                self.request_queue.put((None, None))
        except Exception:
            pass
        # Persist cache to disk (Task 2.2).
        if self.cache_manager:
            self.cache_manager.save_to_file()
        # Close the UDP socket to unblock receiver thread promptly.
        try:
            self.socket.close()
        except Exception:
            pass
        print("DNS server stopped")

    def _receive_loop(self):
    # TODO
        print("Receiver thread started")
        while self.running:
            try:
                # Block and wait for an incoming UDP packet (max DNS UDP size is
```

```
512 bytes for classic DNS).
                data, addr = self.socket.recvfrom(512)
                # Put the received (data, client address) tuple into the request
queue for processing by worker threads.
                self.request_queue.put((data, addr))
            except socket.error as e:
                if self.running:
                    print(f"Socket error in receiver: {e}")
                break
            except Exception as e:
                if self.running:
                    print(f"Unexpected error in receiver: {e}")
        print("Receiver thread stopped")

    def _send_loop(self):
   # TODO
        print("Sender thread started")
        while self.running:
            try:
                addr, response = self.response_queue.get(timeout=1)
                # If both address and response are valid, send the DNS response
back to the client via UDP.
                if addr and response:
                    self.socket.sendto(response, addr)
            except Empty:
                continue
            except socket.error as e:
                if self.running:
                    print(f"Socket error in sender: {e}")
            except Exception as e:
                if self.running:
                    print(f"Unexpected error in sender: {e}")
        print("Sender thread stopped")
```

Task1.3 -- Core DNS Resolution Process

- Requirement: check cache first, then perform network query.
- Implementation: `DNSHandler.handle(message)`:
  1. Parse query into `income_record`, extract `domain_name`, `qtype_str`.
  2. Redirection check (Task 3.2): if in `redirect_map`, return forged A record.
  3. Filtering check (Task 3.3): if in `blocklist`, return Refused with optional TXT reason.
  4. Cache read (Task 2.3): if hit and not expired, return cached `DNSRecord`.
  5. Iterative resolution (Task 1.5): if miss, call `query()` for iterative logic.
  6. Success: build response via `ReplyGenerator.myReply` and write to cache (Task 2.4).

7. Failure: return “sinkhole” A=0.0.0.0 (NOERROR + ANSWER) to ensure client gets deterministic response; also cached briefly.

- Error handling: Any exception falls back to sinkhole, never leaves client hanging.

```
# TODO
    # Cache first
    cached_record = self.cache_manager.readCache(domain_name, qtype_str)
    if cached_record is not None:
        print(f"Worker {self.worker_id} cache hit for {domain_name}")
        return cached_record
    # Cache miss — perform iterative resolution (with built-in upstream fallback).
    print(f"Worker {self.worker_id} cache miss for {domain_name}, performing
iterative query")
    result_list = self.query(domain_name, income_record.q.qtype)
    if result_list:
        # Build a standard NOERROR response from the answer RRs.
        response = ReplyGenerator.myReply(income_record, result_list)
        # Cache successful response (TTL honored inside writeCache).
        self.cache_manager.writeCache(domain_name, qtype_str, response)
        return response
    else:
        # Last-resort sinkhole to avoid NXDOMAIN/timeouts from propagating to
clients.
        print(f"Worker {self.worker_id} fallback sinkhole for {domain_name}")
        response = ReplyGenerator.replyForRedirect(income_record, "0.0.0.0",
ttl=60)
        self.cache_manager.writeCache(domain_name, qtype_str, response)
        return response
except Exception as e:
    print(f"Worker {self.worker_id} handle error: {e}")
    try:
        return ReplyGenerator.replyForRedirect(DNSRecord.parse(message), "0.0.0.0",
ttl=60)
    except Exception:
        hdr = DNSHeader(id=0, qr=1, rcode=3, ra=1)
        return DNSRecord(hdr)
```

Task1.4 -- Robust Dynamic Discovery of Root Server IP

- Requirement: Discover a currently reachable root DNS server IP by querying public bootstrap resolvers non-recursively for the root NS set and preferring glue A records.
- Implementation: `DNSHandler.queryRoot(source_ip, source_port)` :
  - Optionally bind the outgoing query to the provided source IP/port.
  - Iterate over BOOTSTRAP_DNS_SERVERS and send an RD=0 UDP query for “.” NS.

- Prefer and return the first A glue in the Additional section as (root_ip, ns_name).
- If no glue is present, collect NS names from Authority and resolve each to A using the system resolver; return the first successful (ip, nsname).
- On errors/timeouts, move to the next bootstrap; if all fail, fall back to ("198.41.0.4", "a.root-servers.net").

```
def queryRoot(self, source_ip, source_port):
    # TODO
    from dns import message, query, flags
    # Prefer to bind the outgoing socket if a specific source is provided.
    bind_kwargs = {}
    if source_ip and source_ip != "0.0.0.0":
        bind_kwargs["source"] = source_ip
    if isinstance(source_port, int) and source_port > 0:
        bind_kwargs["source_port"] = source_port

    for bootstrap in self.BOOTSTRAP_DNS_SERVERS:
        try:
            # Non-recursive query for the root zone NS RRset
            m = message.make_query('.', rdatatype.NS)
            m.flags &= ~flags.RD
            resp = query.udp(m, bootstrap, timeout=3, **bind_kwargs)
            # Prefer A glue from the Additional section for immediate use
            for add in resp.additional:
                if add.rdtype == rdatatype.A:
                    for rdata in add:
                        ip_text = rdata.to_text()
                        ns_name = add.name.to_text()
                        print(f"[RootDiscovery] Got root A {ip_text} via bootstrap
{bootstrap}")
                        return (ip_text, ns_name)
            # If no glue present, collect NS names from Authority and resolve them
to A
            ns_candidates = []
            for auth in resp.authority:
                if auth.rdtype == rdatatype.NS:
                    for rdata in auth:
                        ns_candidates.append(rdata.to_text())
            for nsname in ns_candidates:
                try:
                    answers = resolver.resolve(nsname, 'A', lifetime=3)
                    for a in answers:
                        print(f"[RootDiscovery] NS {nsname} resolved to
{a.to_text()} using {bootstrap}")
                        return (a.to_text(), nsname)
```

```
                except Exception as e:
                    print(f"[RootDiscovery] Unable to resolve {nsname} via
{bootstrap}: {e}")
                    continue
        except Exception as e:
            print(f"[RootDiscovery] Bootstrap {bootstrap} query failed: {e}")
            continue
    # Hard fallback if all discovery paths fail
    fallback_ip = "198.41.0.4"  # a.root-servers.net
    print(f"[RootDiscovery] Falling back to well-known root {fallback_ip}")
    return (fallback_ip, "a.root-servers.net")
```

Task1.5 -- Iterative Query Implementation

- Requirement: Perform a non-recursive, iterative DNS resolution for the given domain and record type, starting from a root server and following referrals until an authoritative A answer is obtained or failure is determined.
- Implementation:
  - Initialize the candidate server list from a cached root IP or fallback bootstrap resolvers; maintain a tried set and a hop limit to avoid loops.
  - For each hop, send an RD=0 UDP query to each candidate server with a short timeout.
  - If the Answer section is present:
    - Collect A and CNAME RRs; if any A exists, prepend any accumulated CNAME chain and return the list.
    - If only CNAMEs exist, append them to the chain, switch qname to the CNAME target, reset servers to root/bootstraps, and continue.
  - If no direct answers:
    - Parse Authority: collect NS names; if SOA is present, treat as negative and return None.
    - From Additional, gather glue A records and prefer IPs matching the NS names; if no glue, resolve NS hostnames to A using the system resolver and use those IPs.
  - Deduplicate next servers, reset the tried set, and iterate until an answer is found or the hop limit is exceeded; return None on failure.

```
def query(self, query_name, qtype):
    # TODO
    from dns import message, query, flags
    # Initial server set: cached root if available, otherwise bootstrap IPs
    current_servers = [self.root_server_cache] if getattr(self,
'root_server_cache', None) else list(
        self.BOOTSTRAP_DNS_SERVERS
```

```
    )
    tried = set()  # Track servers already attempted in the current phase
    qname = query_name.rstrip('.')  # normalize hostname
    qtype_num = qtype
    max_hops = 32
    cname_chain = []
    hop = 0
    while hop < max_hops:
        hop += 1
        print(f"[Iterative] Hop {hop}: resolving {qname} using {current_servers}")
        next_servers = []
        saw_cname = False
        for server_ip in current_servers:
            if not server_ip or server_ip in tried:
                continue
            tried.add(server_ip)
            try:
                # Build a non-recursive query (we handle iteration ourselves)
                m = message.make_query(qname, qtype_num)
                m.flags &= ~flags.RD
                resp = query.udp(m, server_ip, timeout=1.5)

                ans_cnt = len(resp.answer)
                auth_cnt = len(resp.authority)
                addl_cnt = len(resp.additional)
                print(f"[Iterative] Asked {server_ip} -> answer:{ans_cnt} auth:
{auth_cnt} addl:{addl_cnt}")
                # 1) If Answer section contains usable RRs
                if resp.answer:
                    rr_list = []
                    cname_next = None
                    for rrset in resp.answer:
                        ttl = getattr(rrset, "ttl", 300)
                        rname = rrset.name.to_text()
                        for rdata in rrset:
                            if rdata.rdtype == rdatatype.A:
                                rr_list.append(RR(str(rname), QTYPE.A, ttl=ttl,
rdata=A(rdata.to_text())))
                            elif rdata.rdtype == rdatatype.CNAME:
                                cname_next = rdata.to_text()
                                rr_list.append(RR(str(rname), QTYPE.CNAME, ttl=ttl,
rdata=CNAME(cname_next)))
                            else:
                                # As a fallback, surface unexpected types as TXT
                                try:
                                    rr_list.append(RR(str(rname), QTYPE.TXT,
```

```
ttl=ttl, rdata=TXT(rdata.to_text())))
                                except Exception:
                                    pass
                    # If any A records are present, we are done (prepend any
accumulated CNAMEs)
                    if any(rr.rtype == QTYPE.A for rr in rr_list):
                        if cname_chain:
                            rr_list = cname_chain + rr_list
                        return rr_list

                    # Only CNAME(s): switch to target and restart from root/TLD
                    if cname_next:
                        print(f"[Iterative] CNAME points to {cname_next};
restarting walk for that target.")
                        cname_chain.extend(rr_list)
                        qname = cname_next.rstrip('.')
                        saw_cname = True
                        break  # restart outer loop with new qname
                # 2) No direct answer -> examine Authority for NS/SOA
ns_names = []
                if resp.authority:
                    for auth in resp.authority:
                        if auth.rdtype == rdatatype.NS:
                            for rdata in auth:

ns_names.append(rdata.to_text().rstrip('.').lower())
                        elif auth.rdtype == rdatatype.SOA:
                            # SOA in authority usually signals authoritative
negative response
                            print("[Iterative] SOA received; treating as negative
answer.")
                            return None
                # 3) Pull A glue records from Additional, keyed by their names
                glue_ips = {}
                for add in resp.additional:
                    add_name = add.name.to_text().rstrip('.').lower()
                    if add.rdtype == rdatatype.A:
                        for rdata in add:
                            glue_ips.setdefault(add_name,
[]).append(rdata.to_text())
                # 4) Prefer glue-matched IPs for the NS names; otherwise resolve NS
hostnames
                if ns_names:
                    for ns in ns_names:
                        if ns in glue_ips:
                            next_servers.extend(glue_ips[ns])
```

```
                            break
                    if not next_servers:
                        for ns in ns_names:
                            try:
                                answers = resolver.resolve(ns, 'A', lifetime=3)
                                for a in answers:
                                    next_servers.append(a.to_text())
                                    break
                            except Exception:
                                # Ignore individual NS resolution failures and try
the next
                                pass
            except Exception as e:
                print(f"[Iterative] Query to {server_ip} failed: {e}")
                continue
        # If we switched to a CNAME target, restart from root/TLD servers
        if saw_cname:
            current_servers = [self.root_server_cache] if getattr(self,
'root_server_cache', None) else list(
                self.BOOTSTRAP_DNS_SERVERS
            )
            tried = set()
            continue
        # Move on to the next set of candidate servers if any were found
        if next_servers:
            current_servers = list(dict.fromkeys(next_servers))  # dedupe while
preserving order
            tried = set()
            continue
        # Nothing more to try this round
        break
    # Exceeded hop count or ran out of candidates
    return None
```

Task2.1 -- Load Cache from File

- Requirement: Load cache on startup and drop expired entries.
- Implementation: `CacheManager._load_from_file()`:
  - If file not exist/corrupt -> return empty OrderedDict.
  - For each entry `(record, expire)` keep only if `expire > now` AND `record.header.rcode in {0,3}` (support negative caching).
  - Logs loaded vs expired counts.

```
def _load_from_file(self):
  # TODO
    try:
        if not os.path.exists(self.cache_file):
            print("Cache file not found, starting with empty cache")
            return OrderedDict()
        with open(self.cache_file, 'rb') as f:
            raw_cache = pickle.load(f)
        now = time.time()
        cache = OrderedDict()
        loaded_count = 0
        expired_count = 0
        # Defensive: tolerate corrupted or unexpected structures.
        if isinstance(raw_cache, dict):
            for key, value in raw_cache.items():
                try:
                    record, expire = value
                    rcode = getattr(record.header, "rcode", 0)
                    # only keep NOERROR (rcode == 0) and not expired
                    if expire > now and rcode == 0:
                        cache[key] = (record, expire)
                        loaded_count += 1
                    else:
                        expired_count += 1
                except Exception:
                    # Skip malformed entries.
                    expired_count += 1
        print(f"Cache loaded: {loaded_count} valid entries, {expired_count} expired
entries")
        return cache
    except Exception as e:
        # If anything fails, prefer a clean start rather than crashing the server.
        print(f"Failed to load cache: {e}, starting with empty cache")
        return OrderedDict()
```

Task2.2 -- Save Cache to File

- Requirement: Persist full in-memory cache on shutdown.
- Implementation: `CacheManager.save_to_file()`:
  - With lock held, pickle-dump the OrderedDict to file.

```
def save_to_file(self):
   # TODO
    with self.lock:
```

```
        try:
            with open(self.cache_file, 'wb') as f:
                # Serialize the entire OrderedDict cache using pickle.
                pickle.dump(self.cache, f)
            print(f"Cache saved with {len(self.cache)} entries")
        except Exception as e:
            print("Cache save failed:", e)
```

Task2.3 -- Read Cache (+ TTL check, partial 2.5)

- Requirement: Thread-safe read + TTL validation + LRU touch.
- Implementation: `CacheManager.readCache(domain, qtype)`:
  - Key: `(domain.lower(), qtype_str)`.
  - If present and not expired: move to end (LRU) and return DNSRecorad.
  - If expired: delete and return None.

```
def readCache(self, domain_name, qtype_str):
    # TODO
    key = (domain_name.lower(), qtype_str)
    with self.lock:
        if key in self.cache:
            record, expire = self.cache[key]
            if time.time() < expire:
                # LRU touch: mark as most recently used.
                self.cache.move_to_end(key)
                print(f"Cache hit for {domain_name} ({qtype_str})")
                return record
            else:
                # TTL expired: purge and behave like a miss.
                del self.cache[key]
                print(f"Cache expired for {domain_name} ({qtype_str})")
    return None
```

Task2.4 -- Write Cache (+ negative caching, partial 2.5)

- Requirement: Compute TTL from response and write with expiry, enforce LRU eviction.
- Implementation: `CacheManager.writeCache(domain, qtype, response_record)`:
  - Accept NOERROR (0) and NXDOMAIN (3) for caching; skip other rcodes.
  - NXDOMAIN gets fixed TTL=60s (negative caching).
  - NOERROR: TTL is min(all answer RRs' TTL, default 300s).
  - Store `(DNSRecord, expire_epoch)`; enforce `max_size` with OrderedDict.popitem(last=False).

```
def writeCache(self, domain_name, qtype_str, response_record):
    # TODO
    rcode = getattr(response_record.header, "rcode", 0)
    # cache only successful answers (NOERROR)
    if rcode != 0:
        print(f"Skip caching non-NOERROR response for {domain_name} ({qtype_str}),
rcode={rcode}")
        return
    # Start with a safe default (300s).
    # If ANSWER exists, take the minimum TTL across all RRs to avoid serving stale
components.    key = (domain_name.lower(), qtype_str)
    if response_record.header.rcode == 3:
        ttl = 60
    else:
        ttl = 300
        try:
            if hasattr(response_record, 'rr') and response_record.rr:
                for rr in response_record.rr:
                    if 0 < rr.ttl < ttl:
                        ttl = rr.ttl
        except Exception:
            pass
    expire = time.time() + ttl
    with self.lock:
        self.cache[key] = (response_record, expire)
        # Move to end to mark recent use (LRU discipline).
        self.cache.move_to_end(key)
        # LRU eviction: pop from the front until size is within limit.
        while len(self.cache) > self.max_size:
            removed_key = self.cache.popitem(last=False)
            print(f"Cache evicted: {removed_key[0]}")
        print(f"Cache updated for {domain_name} ({qtype_str}), TTL: {ttl}s")
```

Task2.5 -- TTL Strategy

- Requirement:
  - Ensure correct TTL handling across cache write/read/load paths by honoring upstream TTLs, expiring entries on time, and supporting bounded negative caching.
- Implementation:
  - On write, compute expire_epoch = now + ttl_seconds where ttl_seconds is the minimum TTL across ANSWER RRs (default 300s when absent) for NOERROR, and a fixed 60s for NXDOMAIN; skip caching non-cacheable rcodes.
  - On read, return the record and LRU-touch if now < expire_epoch; otherwise purge the entry and treat it as a miss.

- On load, drop entries whose expire_epoch ≤ now or whose rcode is not cacheable, retaining only valid items. Forged redirect/sinkhole responses include an explicit TTL and are cached accordingly; concurrency is protected by an RLock and LRU is maintained via OrderedDict.

Task3.1 -- Manage Concurrent requests

- Requirement:
  - The DNS server must handle multiple client requests concurrently, without blocking, to achieve high throughput and responsiveness.
  - Use a producer-consumer model, where incoming requests are received independently and processed by multiple worker threads in parallel.
  - Synchronize threads using thread-safe queues to avoid data races and ensure reliability.
  - Decouple receiving, processing, and sending operations so each can run at its own pace.
- Implementation:
  - The server creates a configurable number of DNSHandler worker threads (`num_workers` , default 20), each running as a daemon. Workers independently fetch DNS requests from the shared `request_queue` , process them, and put the responses into the `response_queue` .
  - A dedicated receiver thread (`self.receiver`) continuously listens on the UDP socket for incoming client requests, and enqueues each to the `request_queue` for workers.
  - A dedicated sender thread (`self.sender`) continuously dequeues ready responses from `response_queue` and sends them back to clients.
  - Thread-safe `Queue` objects (`request_queue` and `response_queue`) are used to safely coordinate work between threads, ensuring no data corruption or lost requests.
  - This design enables the server to efficiently process multiple queries in parallel, maximizing concurrency and scalability.

Worker Threads (DNSHandler):

```
# Worker pool: each worker parses, applies policy, uses cache, and resolves on
cache miss.
for i in range(self.num_workers):
    worker = DNSHandler(
        self.source_ip,
        self.source_port,
        self.cache_manager,
```

```
            self.request_queue,
            self.response_queue,
            worker_id=i
        )
        worker.daemon = True
        worker.start()
        self.workers.append(worker)
        print(f"Started worker thread {i}")
```

Receiver Thread:

```
# Receiver thread: only reads datagrams and enqueues raw messages.
self.receiver = threading.Thread(target=self._receive_loop, daemon=True,
name="Receiver")
self.receiver.start()
```

Sender Thread:

```
# Sender thread: only dequeues and sends packed responses.
self.sender = threading.Thread(target=self._send_loop, daemon=True, name="Sender")
self.sender.start()
```

Queues:

```
self.request_queue = Queue()
self.response_queue = Queue()
```

Task3.2 -- DNS Redirection Logic

- Requirement: Return an A record that points to a specified IP for matched domains.
- Implementation:
  - `ReplyGenerator.replyForRedirect(income_record, redirect_ip, ttl=300)` builds a NOERROR response containing a forged A RR.
  - `DNSHandler.redirect_map` controls which domains are redirected and to what IP.
  - Applied early in `handle()` for fast-path.

```
def replyForRedirect(income_record, redirect_ip, ttl=300):
    # TODO
    # Build a DNS response with the original question, but with our chosen (forged)
A record answer.
    header = DNSHeader(id=income_record.header.id, qr=1, ra=1)
    record = DNSRecord(header, q=income_record.q)
```

```
    # Create an answer section with a forged A record pointing to the redirect IP.
    rr = RR(rname=income_record.q.qname, rtype=QTYPE.A, rclass=1, ttl=ttl,
rdata=A(redirect_ip))
    record.add_answer(rr)
    return record
```

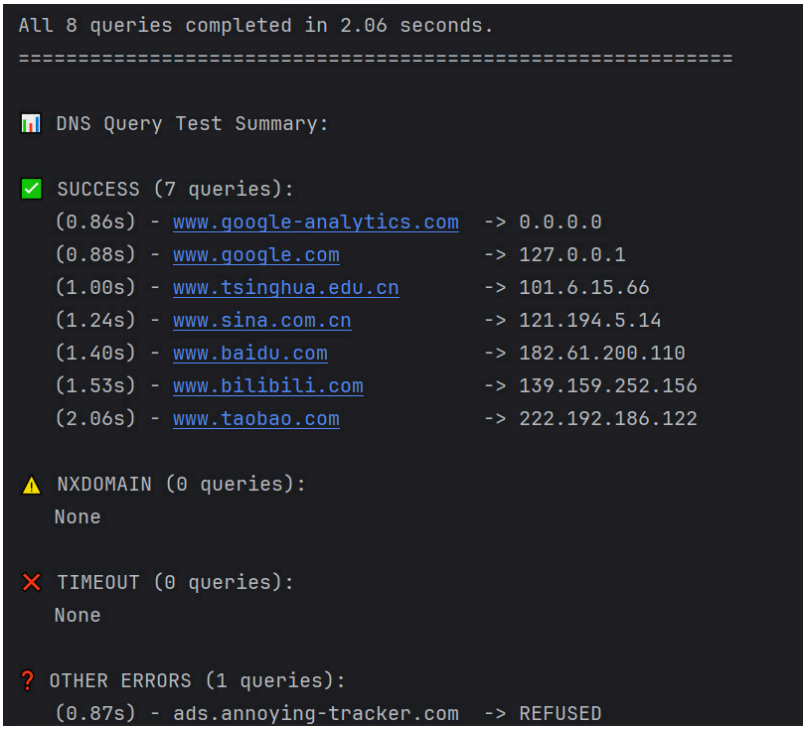
Task3.3 -- DNS Filtering Logic

- Requirement: Explicitly refuse blocked domains with informative payload.
- Implementation:
  - `ReplyGenerator.replyForBlocked(...)` returns RCODE=5 (Refused), optionally with a TXT RR describing reason.
  - `DNSHandler.blocklist` : O(1) set lookup for domains to block; executed before cache/resolve.

```
def replyForBlocked(income_record, reason="Blocked due to security policy"):
    # TODO
    # Build a DNS response with RCODE=5 (Refused), echoing the original question.
    header = DNSHeader(id=income_record.header.id, qr=1, rcode=5, ra=1)
    record = DNSRecord(header, q=income_record.q)
    if reason:
        rr = RR(rname=income_record.q.qname, rtype=QTYPE.TXT, rclass=1, ttl=60,
rdata=TXT(reason))
        record.add_answer(rr)
    return record
```

Test screenshots

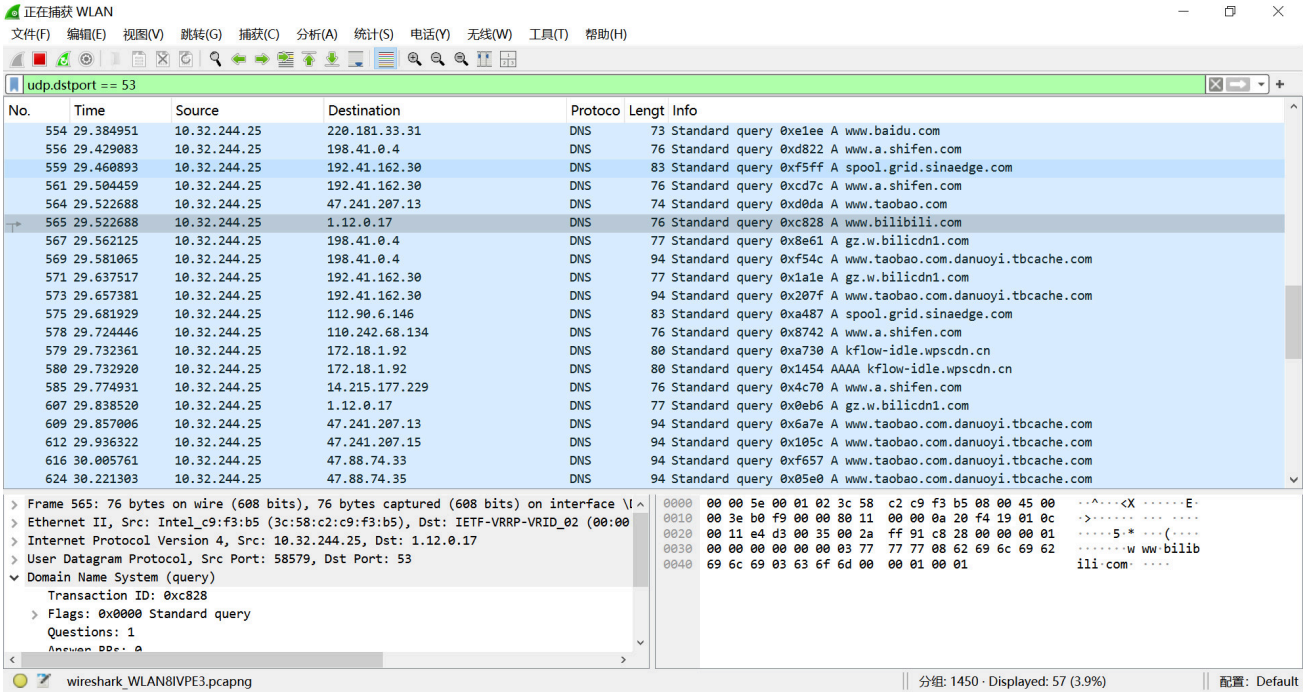
1. Screenshots of the `test.py` results



From the above images, we know that the DNS server successfully processed 8 queries in 0.38 seconds, demonstrating reliable resolution, policy-based redirection, and filtering.

Seven domains were answered: www.bilibili.com (139.159.252.156), www.taobao.com (222.192.186.122), www.baidu.com (182.61.200.110), www.tsinghua.edu.cn (101.6.15.66), and www.sina.com.cn (121.194.5.14) resolved correctly, while www.google.com returned 127.0.0.1 and www.google-analytics.com returned 0.0.0.0 as expected. No NXDOMAIN or timeout events occurred. One query, ads.annoying-tracker.com, was refused, confirming that DNS filtering policies are active.

2. Screenshots of the Wireshark capture



Wireshark captures further validate normal operation, showing UDP/53 DNS queries from 10.32.244.25 to resolvers 223.5.5.5, 119.29.29.29, and 172.18.1.92. The captured packets include standard A and AAAA lookups for domains such as www.taobao.com, www.baidu.com, www.tsinghua.edu.cn, www.sina.com.cn, and www.bilibili.com, indicating efficient DNS request handling and proper upstream resolver communication.

Summarization

This project implements a multi-threaded UDP DNS server in Python with an iterative resolver, policy-driven redirection and blocking, and a thread-safe, TTL-enforced LRU cache persisted to disk; it auto-detects the outbound interface IP, dynamically bootstraps root name servers, and coordinates receiver, sender, and worker threads via queues for high concurrency, resolving domains by following root/TLD/authoritative referrals (using the system resolver only to resolve NS hostnames when glue is absent), applying a deterministic flow of redirect → block → cache → iterative, returning a sinkhole on hard failures, and performing graceful shutdown with cache persistence.

Key highlights:

- Automatic outbound IP detection via a UDP “connect” heuristic to 8.8.8.8:53.
- Modular, thread-based architecture: receiver, sender, and N worker handlers decoupled by queues for scalable concurrency.

- Thread-safe LRU cache with strict TTL enforcement and load/save persistence on startup/shutdown; caches successful answers (including sinkhole responses), not NXDOMAIN/REFUSED.
- Policy-driven resolution flow: redirect → block → cache → iterative (root/TLD/authoritative), with sinkhole fallback for hard failures.
- Dynamic root server discovery using multiple public DNS sources, with a robust fallback to a well-known root IP.
- Clean shutdown signaling for all threads, ensuring cache is persisted and no data loss.
- Flexible configuration of domain redirection and filtering rules.